

## Contrôle de connaissance

Unité 4CS02 – Cryptologie  
Durée : 1h30

*Les supports de cours ne sont pas autorisés.*

*Une page recto-verso manuscrite ou imprimée est autorisée.*

*Les dispositifs électroniques sont interdits.*

*Les réponses peuvent être rédigées en anglais.*

*La rédaction et la justification des réponses seront prises en compte dans la note.*

*Le barème est donné à titre indicatif.*

## 1 Un chiffrement par bloc

L'algorithme PRESENT est un système de chiffrement par blocs standardisé à l'ISO/IEC. Par rapport à AES, il nécessite beaucoup moins de lignes de code, et il est ainsi parfois utilisé dans le contexte de l'Internet des Objets, dans lequel certains terminaux disposent de peu de puissance de calcul.

La Figure 1 représente la structure globale de l'algorithme PRESENT en chiffrement, avec un nombre de tours égal à 31.

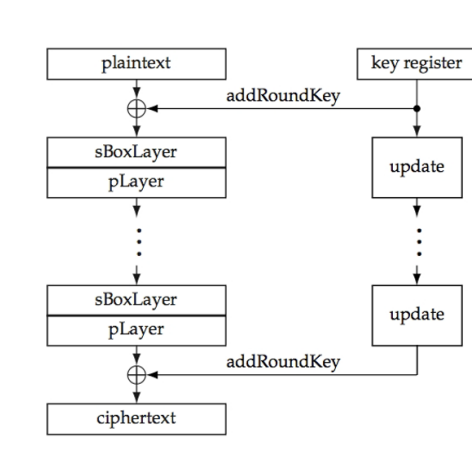


Figure 1: Structure de PRESENT

Comme vous pouvez le voir, cette structure utilise notamment une fonction **pLayer** décrite formellement par la fonction  $P : \{0, \dots, 63\} \longrightarrow \{0, \dots, 63\}$  qui se définit de la façon suivante :

$$P(i) = \begin{cases} i \cdot 16 \pmod{63} & \text{si } i \in \{0, \dots, 62\} \\ 63 & \text{si } i = 63 \end{cases}$$

### Question 1 (2 points)

- Quel est le PGCD de 16 et 63 ? En déduire que 16 a un inverse modulo 63. Que souhaiteriez-vous utiliser comme méthode pour trouver cet inverse ? Quel est cet inverse ?
- En utilisant la question précédente, montrer que pour tout couple  $(i, j) \in \{0, \dots, 62\}^2$ , si  $i \cdot 16 = j \cdot 16 \pmod{63}$ , alors  $i = j$ .

#### Corrigé question 1.

- $\text{PGCD}(16, 63) = 1$ . Il existe donc  $u$  et  $v$  tels que  $16 \cdot u + 63 \cdot v = 1$ . En prenant cette équation modulo 63, on obtient que  $16 \cdot u = 1 \pmod{63}$  et donc que  $u$  est l'inverse de 16 modulo 63. Pour trouver  $u$ , il faut utiliser l'algorithme d'Euclide étendu. Cela donne  $u = 4$  et  $v = -1$ . Donc l'inverse de 16 modulo 63 est 4.
- Pour tout  $(i, j) \in \{0, \dots, 62\}^2$ , si  $i \cdot 16 = j \cdot 16 \pmod{63}$ , alors  $4 \cdot i \cdot 16 = 4 \cdot j \cdot 16 \pmod{63}$  (en multipliant chaque côté de l'égalité par 4), et comme, d'après la question précédente, 4 est l'inverse de 16 modulo 63,  $4 \cdot 16 = 1 \pmod{63}$ , et donc  $i = j \pmod{63}$ . Comme  $i$  et  $j$  sont dans  $\{0, \dots, 62\}$ , nous obtenons  $i = j$ .

En combinant ce dernier résultat avec le fait que  $P(63) = 63$  et que les ensembles de départ et d'arrivée de  $P$  sont égaux, cela démontre que  $P$  est une bijection, et donc une permutation.

### Question 2 (1 points)

- Que reconnaissez-vous dans la structure de la Figure 1 ?
- En vous appuyant sur les conclusions obtenues ci-dessus sur `pLayer`, quelles propriétés doit vérifier la fonction `sBoxLayer` ?

#### Corrigé question 2.

- On reconnaît les principes de Shannon, avec plusieurs tours comprenant un xor (cf. `addRoundKey`) avec une clé venant du `key register`, puis une étape de substitution (`sBoxLayer`) et une étape de permutation (`pLayer`).
- Comme dit précédemment, `sBoxLayer` doit être une fonction de substitution non linéaire.

Une version de PRESENT utilise des messages de 64 bits et des clés de 80 bits. Sébastien souhaiterait augmenter la taille des messages à 128 bits et la taille de la clé à 160 bits en utilisant le mécanisme décrit dans la Figure 2, où  $IV$  est une valeur aléatoire, la clé  $K = K_1 \| K_2$  est composée de deux clés ( $K_1$  et  $K_2$ ) de l'algorithme PRESENT (donc 80 bits chacune), et le message  $M = M_1 \| M_2$  est composé de deux parties ( $M_1$  et  $M_2$ ) de 64 bits chacune. Le chiffré final est formé de la façon suivante :  $IV \| C_1 \| C_2$ .

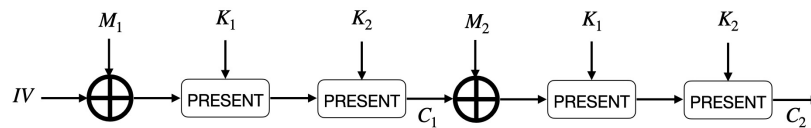


Figure 2: Structure proposée par Sébastien

### Question 3 (3 points)

- Pourquoi Sébastien souhaite-t-il augmenter la taille de la clé ? Quelle attaque est aujourd'hui considérée comme possible sur une taille de clé de 80 bits ?

- *A quoi vous fait penser la structure décrite dans la Figure 2 ? Quel est son usage majeur aujourd'hui ?*
- *Que pensez-vous de l'idée de Sébastien, en justifiant votre réponse ?*

### Corrigé question 3.

- L'ANSSI et la plupart des agences de sécurité préconisent aujourd'hui des tailles de clé de 128 bits. Une attaque par recherche exhaustive (ou force brute, i.e., tester toutes les clés possibles) est aujourd'hui considérée comme plausible avec une taille de 80 bits.
- La structure proposée fait penser au mode CBC. Il est aujourd'hui utilisé avec un système de chiffrement par blocs pour permettre de chiffrer des messages de taille quelconque (assurant la confidentialité), ou pour réaliser un MAC (intégrité et authenticité).
- Sachant qu'il cherche à faire du chiffrement, un attaquant peut avoir accès à  $IV$ ,  $C_1$  et  $C_2$ . Or, si je prends par exemple  $C_1$  et  $C_2$ , le schéma proposé correspond à un double PRESENT, avec deux clés différentes. Il peut donc utiliser la même attaque (*meet in the middle*) que celle vue en cours sur le double DES. Cela ne donne donc pas la sécurité escomptée puisque l'utilisation d'une clé de 1660 bits ne donne pas une sécurité de 160 bits ! Le fait qu'il y ait un xor entre le chiffré  $C_2$  et le message  $M_2$  ne pose pas de problème ici, puisque dans cette attaque, nous considérons que l'attaquant a accès au message. C'est en effet une attaque à texte clair connu.

Une autre version de PRESENT utilise des messages de 64 bits et des clés de 128 bits.

### Question 4 (2 points)

- *Est-ce que ces paramètres sont aujourd'hui compatibles avec les règles de l'ANSSI ? Avec quelles contraintes potentiellement ? Quel peut être l'intérêt pour les concepteurs de PRESENT d'avoir considéré une telle taille de bloc ?*
- *Vous venez d'apprendre qu'il existe une attaque en  $2^{127}$  opérations, et nécessitant  $2^{19}$  données, sur un PRESENT complet (c'est-à-dire sur les 31 tours). Est-ce que vous pensez que cette attaque pose problème ?*

### Corrigé question 4.

- Non, ces paramètres ne sont pas compatibles avec les règles de l'ANSSI. Il faut des tailles de bloc de 128 bits. L'intérêt réside, pour les concepteurs, dans le fait qu'il pourrait souhaiter faire un schéma qui aille très vite, et la gestion de messages plus longs peut ralentir l'exécution cette dernière. Par contre, dans le cas où il ne faudrait chiffrer que des messages de moins de 64 bits (dans le contexte de l'IoT par exemple), cette contrainte pourrait être moins forte. Et l'ANSSI n'empêche pas de tels schémas d'exister. Elle donne des préconisations.
- Non, cette attaque n'est pas très gênante en pratique parce qu'elle ne perd qu'un seul bit de sécurité par rapport à l'attaque par force brute. Cependant, cela peut montrer une faiblesse dans ce schéma, dont il reste à surveiller, d'autant que le nombre de données ( $2^{19}$ ) est relativement faible.

## 2 Echange de clé authentifié

Nous nous intéressons maintenant à deux interlocuteurs (Alice et Bob) qui souhaitent communiquer entre eux de façon confidentielle, intègre et authentifiée. Ils ne possèdent a priori aucun secret en commun. Sébastien souhaite qu'ils utilisent le schéma décrit de la façon suivante.

1. Bob envoie sa clé publique RSA  $pk_B$  à Alice.
2. Alice génère une clé cryptographique  $K$ .
3. Alice chiffre la clé  $K$  à l'aide de l'algorithme RSA en utilisant la clé publique  $pk_B$  de Bob.
4. Alice envoie le chiffré  $c$  ainsi obtenu à Bob.
5. Bob déchiffre  $c$  à l'aide de sa clé privée  $sk_B$  afin d'obtenir  $K$ .
6. Alice et Bob utilisent  $K$  pour sécuriser leurs communications.

**Question 5 (2 points)**

- Décrivez la ou les faiblesses de ce protocole.
- Que faut-il faire pour améliorer sa sécurité ?

**Corrigé question 5.**

- Alice ne peut pas vérifier que la clé publique utilisée provient bien de Bob. Par ailleurs, Bob ne peut pas s'assurer que le message chiffré qu'il reçoit provient bien d'Alice. Il n'y a donc aucune authentification dans cette proposition.
- Il faudrait que Bob envoie son certificat (généré et signé par une autorité de confiance) à Alice en même temps que sa clé publique. Il faudrait aussi qu'Alice produise une signature numérique de son envoi à Bob (avec aussi son certificat numérique).

Nous nous intéressons maintenant à la sécurisation des communications (étape 6ci-dessus). Sébastien souhaiterait utiliser RSA pour la confidentialité, l'intégrité et l'authentification.

**Question 6 (3 points)**

- Est-ce que RSA peut effectivement être utilisé pour répondre à ces trois besoins ? Est-ce une bonne idée d'utiliser RSA ? Justifiez vos réponses.
- Si à la place, nous n'utilisons que de la cryptographie à clé secrète pour répondre à ces trois besoins, quel outil cryptographique faut-il utiliser pour chacun de ces trois besoins ?
- Donnez le nom de modes opératoires que nous pourrions utiliser pour mettre en œuvre ces outils cryptographiques (les détails des modes opératoires ne sont pas demandés), ainsi que la façon de les utiliser côté Alice et côté Bob. Est-ce que l'algorithme *PRESENT* est compatible avec ces modes opératoires ?

**Corrigé question 6.**

- RSA pourrait effectivement être utilisé pour répondre à ces trois besoins. Il faudrait utiliser RSA avec OAEP pour la confidentialité et RSA avec PSS pour l'intégrité et l'authentification. Par contre, ce n'est pas forcément une bonne idée, parce que RSA ne sera pas du tout approprié pour gérer des très gros messages. Cela pourrait être possible pour la signature, mais l'efficacité en serait affectée.
- Pour la confidentialité, il faut utiliser un système de chiffrement. Pour l'intégrité et l'authenticité, il faut utiliser un MAC (code d'authentification de message).
- Pour le chiffrement, on peut utiliser le mode CTR (ou le mode CBC). Pour l'intégrité et l'authenticité, on peut utiliser le mode CBC. Dans ce cas, il faut utiliser le *Encrypt-then-MAC*. Il est aussi possible d'utiliser d'autres modes opératoires (CCM, GCM, EAX) qui vont fournir confidentialité, intégrité et authenticité en un seul mode. L'algorithme *PRESENT* étant un système par bloc, il est tout à fait compatible avec ces modes opératoires.

### 3 Etude de RSA

Pour exécuter le système RSA, Sébastien choisit d'utiliser l'algorithme décrit par Algorithm 1.

---

**Algorithm 1** Algorithme à étudier

---

```
1: procedure FUNCTION( $a, b, m$ )
2:    $r \leftarrow 1$ 
3:    $a \leftarrow a \pmod{m}$ 
4:   while  $b > 0$  do
5:     if  $b \pmod{2} == 1$  then
6:        $r \leftarrow (r \cdot a) \pmod{m}$ 
7:        $b \leftarrow b/2$ 
8:        $a \leftarrow (a \cdot a) \pmod{m}$ 
9:   return  $r$ 
```

---

**Question 7 (3 points)**

- Que fait cet algorithme ? Rajoutez des commentaires pour expliquer les étapes importantes.
- A quelle(s) étape(s) de RSA cet algorithme est-il pertinent ?
- Sans aucun traitement du message avant l'exécution de RSA (le chiffrement du message  $m$  est  $c = m^e \pmod{n}$  avec les notations usuelles), quel est le niveau de sécurité atteint ? Sur quel problème difficile s'appuie-t-on pour atteindre un tel niveau de sécurité ? D'après l'ANSSI, quelle est aujourd'hui la taille des clés à utiliser avec RSA ?

**Corrigé question 7.**

- Cet algorithme permet de calculer une exponentiation modulaire, i.e.,  $a^b \pmod{m}$ .  $b$  est l'exposant et à chaque tour, on consomme un bit de  $b$ . Si le  $i$ -ème bit correspondant est 1 (ligne 5), alors on multiplie le résultat  $r$  par  $a^i$  (ligne 6). On passe au bit suivant par la ligne 7, et on calcule le  $a^{i+1}$  pour une éventuelle utilisation lors de la boucle suivante (ligne 8).
- Cet algorithme va être utilisé lors du chiffrement, du déchiffrement, mais aussi pour signer et pour vérifier une signature.
- On atteint la *one-wayness* avec texte clair choisi (OW-CPA) avec le RSA de base. Cette sécurité repose sur le problème RSA (extraction de la racine  $e$ -ème). Si je sais résoudre efficacement le problème de la factorisation, alors ce problème RSA est considéré comme facile. Aujourd'hui, l'ANSSI demande des clés de 3072 bits (taille du module).

Afin d'obtenir l'indistingabilité de RSA, Sébastien préconise d'utiliser la procédure suivante en chiffrement (le message est noté  $m$ ), où  $\alpha$  est une valeur fixe, et  $\mathcal{G}$  et  $\mathcal{H}$  sont deux fonctions de hachage.

1. Calculer  $v = \mathcal{G}(\alpha)$ .
2. Calculer  $s = m \oplus v$ .
3. Calculer  $w = \mathcal{H}(v)$ .
4. Calculer  $t = w \oplus \alpha$ .
5. Exécuter RSA sur le message  $s||t$  pour obtenir  $c$ .
6. Envoyer  $c$ .

### Question 8 (4 points)

- Expliquez, avec des détails, pourquoi Sébastien n'obtient pas l'indistingabilité désirée.
- Sébastien se rend compte de son erreur et décide de choisir un  $\alpha$  à chaque fois différent, et de renvoyer  $(c, \alpha)$ . Est-ce mieux ?
- Qu'aurait dû faire Sébastien ?
- Expliquez comment doit se dérouler le déchiffrement.
- La structure proposée correspond à un schéma de Feistel à deux tours. Est-ce que l'attaque sur un schéma de Feistel à deux tours que nous avons vu en TD pose problème ici ? Expliquez votre réponse.

### Corrigé question 8.

- Cet exercice sera vu en TD.